

Coding Challenge — Concurrent Balance Service

Senior Java Developer

زمان پیشنهادی

۴ تا ۶ ساعت

هدف

در این Challenge باید یک سرویس ساده برای مدیریت موجودی حساب کاربران طراحی و پیاده‌سازی کنید. سرویس شما باید بتواند در شرایطی که تعداد زیادی درخواست به‌صورت هم‌زمان دریافت می‌شوند، موجودی حساب‌ها را درست، پایدار و قابل اتکا مدیریت کند.

تمرکز اصلی Challenge روی موارد زیر است:

- Correctness
- Concurrent processing
- Data consistency
- Idempotency
- کیفیت طراحی
- تست‌پذیری
- تصمیم‌های مهندسی و Trade-off ها

تعداد Feature هایی که پیاده‌سازی می‌کنید، به‌اندازه کیفیت راهکار و تصمیم‌های فنی شما اهمیت ندارد.

سناریو

فرض کنید یک سیستم مالی داریم که برای هر کاربر یک Account با موجودی مشخص نگهداری می‌کند.

سرویس باید عملیات زیر را پشتیبانی کند.

1. Credit

افزایش موجودی یک حساب:

```
credit(accountId, amount, transactionId)
```

مثال:

Initial Balance: 1,000

Credit: 500

Final Balance: 1,500

2. Debit

کاهش موجودی یک حساب در صورتی که موجودی کافی باشد:

```
debit(accountId, amount, transactionId)
```

اگر موجودی حساب برای انجام عملیات کافی نباشد، عملیات نباید انجام شود.

مثال:

Initial Balance: 1,000

Debit: 700

Final Balance: 300

اما:

Initial Balance: 1,000

Debit: 1,200

Operation: Failed

Final Balance: 1,000

3. Transfer

انتقال مبلغ از یک حساب به حساب دیگر:

```
transfer(  
    sourceAccountId,  
    destinationAccountId,  
    amount,  
    transactionId  
)
```

مثال:

Account A: 1,000
Account B: 500

Transfer 300 from A to B

Account A: 700
Account B: 800

انتقال باید **Atomic** باشد.

یعنی نباید حالتی ایجاد شود که مبلغ از حساب مبدأ کم شده باشد اما به حساب مقصد اضافه نشده باشد.

API

پیاده‌سازی HTTP API الزامی نیست.

حداقل باید یک Interface مشابه زیر در پروژه وجود داشته باشد:

```
public interface BalanceService {  
  
    void credit(  
        String accountId,  
        long amount,  
        String transactionId  
    );  
  
    void debit(  
        String accountId,  
        long amount,  
        String transactionId  
    );  
  
    void transfer(  
        String sourceAccountId,  
        String destinationAccountId,  
        long amount,  
        String transactionId  
    );  
  
    long getBalance(String accountId);  
}
```

در صورت تمایل می‌توانید این Interface را به صورت REST API نیز ارائه کنید.

Requirements

1. Concurrency

ممکن است چندین درخواست به صورت همزمان روی یک Account اجرا شوند.

برای مثال:

```
Initial Balance = 1,000
```

```
Thread 1 → debit(A, 700)
```

```
Thread 2 → debit(A, 700)
```

در این شرایط فقط یکی از عملیات باید موفق شود.

سیستم نباید در شرایط Concurrent به موارد زیر منجر شود:

- Negative Balance
- Lost Update
- Incorrect Balance
- اجرای ناقص یک عملیات

همچنین عملیات روی حساب‌های مستقل نباید بدون دلیل یکدیگر را Block کنند.

برای مثال:

```
Thread 1 → Account A
```

```
Thread 2 → Account B
```

باید بتوانند تا حد امکان به صورت مستقل پردازش شوند.

2. Idempotency

ممکن است یک درخواست به دلیل Retry چندین بار ارسال شود.

transactionId شناسه یکتای یک عملیات مالی است.

برای مثال:

```
credit(A, 500, "TX-100")
```

```
credit(A, 500, "TX-100")
```

```
credit(A, 500, "TX-100")
```

هر سه درخواست مربوط به یک Transaction هستند و نباید باعث افزایش 1,500 واحدی موجودی شوند.

در نهایت فقط یک بار باید اثر Transaction روی موجودی اعمال شود:

Initial Balance = 1,000

Final Balance = 1,500

همین رفتار باید برای موارد زیر نیز برقرار باشد:

- Credit
- Debit
- Transfer

نکته مهم: ممکن است درخواست‌های تکراری به صورت همزمان دریافت شوند.

3. Consistency

در هر لحظه باید Invariant های اصلی سیستم برقرار باشند.

از جمله:

- موجودی حساب نباید بدون یک عملیات معتبر تغییر کند.
- Debit نباید باعث Negative Balance شود.
- یک Transaction نباید بیش از یک بار روی موجودی اعمال شود.
- Transfer باید اثر یکپارچه‌ای روی حساب مبدأ و مقصد داشته باشد.
- مبلغ کسر شده از مبدأ و مبلغ اضافه شده به مقصد باید دقیقاً یکسان باشد.

4. Validation

حداقل موارد زیر باید اعتبارسنجی شوند:

Invalid Amount

مبلغ Transaction باید معتبر باشد.

برای مثال:

$\text{amount} \leq 0$

باید رد شود.

Unknown Account

در صورتی که Account وجود نداشته باشد، عملیات باید با خطای مناسب مواجه شود.

Same Account Transfer

رفتار سیستم در شرایط زیر باید مشخص و منطقی باشد:

```
transfer(  
    "A",  
    "A",  
    100,  
    "TX-100"  
)
```

تصمیم خود را در README توضیح دهید.

5. Transfer Atomicity

فرض کنید:

```
Account A = 1,000  
Account B = 500
```

و درخواست زیر اجرا می‌شود:

```
transfer(A, B, 300, "TX-1")
```

نتیجه معتبر:

```
A = 700  
B = 800
```

اما سیستم نباید هیچ‌گاه در شرایط عادی به وضعیتی مانند زیر برسد:

```
A = 700  
B = 500
```

یا:

```
A = 1,000  
B = 800
```

همچنین اگر اجرای Transaction با خطا مواجه شود، وضعیت نهایی سیستم باید با semantics طراحی شما سازگار باشد و نباید یک Transfer ناقص ایجاد شود.

نحوه رسیدن به این رفتار را خودتان طراحی کنید.

Testing

تست‌ها بخش مهمی از Challenge هستند.

Idempotency Tests

مثلاً:

```
Initial Balance = 1,000  
  
Credit 100 with TX-1  
Credit 100 with TX-1  
Credit 100 with TX-1
```

انتظار:

```
Final Balance = 1,100
```

همچنین Idempotency مربوط به Debit و Transfer را تست کنید.

Concurrent Tests

حداقل یک سناریوی Concurrent برای یک Account و یک سناریو برای چند Account طراحی کنید.

برای مثال:

```
Initial Balance = 100,000  
  
1,000 concurrent operations  
  
Expected Final Balance = ...
```

تست باید بتواند مشخص کند که راهکار شما در شرایط Concurrent همچنان Correct است.

در صورت امکان، تستی طراحی کنید که احتمال وقوع Race Condition را افزایش دهد.

Technology

پیدادهسازی باید با:

- **Java 21**
- **Gradle یا Maven**
- **Spring Boot**

انجام شود.

استفاده از موارد زیر نیز اختیاری است:

- Database
- Redis
- Kafka
- Docker
- سایر Infrastructure ها

در صورتی که از هرکدام از این تکنولوژی‌ها استفاده کردید، در README توضیح دهید:

1. چرا از آن استفاده کردید؟
2. چه مسئله‌ای را حل می‌کند؟
3. Trade-off هایی ایجاد می‌کند؟

لازم نیست صرفاً برای پیچیده‌تر کردن پروژه از Infrastructure های اضافی استفاده کنید.

یک راهکار ساده و خوب طراحی شده کاملاً قابل قبول است.

Project Structure

ساختار پروژه و نحوه سازمان‌دهی کد به انتخاب شماست.

حداقل انتظار می‌رود کد از بخش‌های منطقی مانند موارد زیر تشکیل شده باشد:

```
src/  
├── main/  
│   └── java/  
├── test/  
│   └── java/
```

استفاده از Architecture یا Design Pattern خاصی الزامی نیست.

README

همراه پروژه یک README.md ارائه کنید.

README باید به صورت مختصر توضیح دهد:

Architecture

ساختار کلی راهکار شما چیست؟

اجزای اصلی سیستم چه هستند؟

Concurrency

چطور Thread Safety را تضمین کرده‌اید؟

چرا این روش را انتخاب کرده‌اید؟

در شرایط Concurrent چه اتفاقی برای عملیات روی یک Account می‌افتد؟

Idempotency

چطور از اجرای چندباره یک Transaction جلوگیری می‌کنید؟

اگر یک Transaction به صورت همزمان چند بار دریافت شود چه اتفاقی می‌افتد؟

Transfer

چطور Atomicity مربوط به Transfer را تضمین کرده‌اید؟

آیا در طراحی شما احتمال Deadlock وجود دارد؟

اگر بله، چه شرایطی باعث آن می‌شود؟

اگر خیر، چگونه از آن جلوگیری کرده‌اید؟

Submission

پروژه را به صورت یک Git Repository تحویل دهید.

Repository باید شامل موارد زیر باشد:

Source Code
Tests
README.md
Build Configuration

پروژه باید با یک دستور استاندارد Build و Test شود.

برای مثال:

```
./gradlew test
```

یا:

```
./mvnw test
```

در صورتی که برای اجرای پروژه یا تست‌ها Dependency یا Configuration خاصی نیاز است، آن را در README توضیح دهید.

مهم‌تر از همه

این Challenge برای ارزیابی توانایی شما در حل مسئله و تصمیم‌گیری مهندسی طراحی شده است.

لازم نیست همه چیز را کامل کنید.

اگر بین اضافه کردن Feature های بیشتر و بهبود Correctness، Concurrency، Testing یا Design مجبور به انتخاب شدید، اولویت با کیفیت راهکار است.

ما بیشتر به موارد زیر توجه می‌کنیم:

- آیا راهکار شما Correct است؟
- آیا در شرایط Concurrent رفتار قابل پیش‌بینی دارد؟
- آیا Idempotency به درستی پیاده‌سازی شده است؟
- آیا Transfer به شکل قابل اتکا انجام می‌شود؟
- آیا تست‌های مناسبی برای سناریوهای حساس نوشته‌اید؟
- آیا می‌توانید تصمیم‌های فنی خود را توضیح دهید؟
- آیا محدودیت‌ها و Trade-off های راهکار خود را می‌شناسید؟

استفاده از AI

استفاده از AI Coding Assistants در طول Challenge مجاز است.

با این حال، کدی که تحویل می‌دهید باید کاملاً برای شما قابل درک باشد.

در مرحله Technical Review ممکن است درباره موارد زیر از شما سوال شود:

- دلیل انتخاب Design
- نحوه مدیریت Concurrency
- نحوه تضمین Idempotency
- رفتار سیستم در شرایط Failure
- Performance Characteristics
- Trade-off های راهکار

بنابراین انتظار می‌رود بتوانید کد و تصمیم‌های فنی خود را توضیح داده و از آن‌ها دفاع کنید.

زمان پیشنهادی

۴ تا ۶ ساعت

در صورتی که به دلیل محدودیت زمانی نتوانستید همه قسمت‌ها را پیاده‌سازی کنید، مشکلی نیست.

در این شرایط در README توضیح دهید:

- چه قسمت‌هایی را پیاده‌سازی کرده‌اید؟
- چه قسمت‌هایی باقی مانده است؟
- اگر زمان بیشتری داشتید، چگونه ادامه می‌دادید؟

Good engineering decisions are more important than completing every possible feature ☺